

Module 2 — Unit Test Frameworks

Student Name	Shreya Ghosh
Course	B.Tech — Computer Science and Business Systems (CSBS) (4th Year)
Enrollment No.	12023052018010
Programming Language	Python
Automation Tool	Selenium WebDriver
Academic Year	2026

Experiment 7 — Page Object Model (POM) Restructure

1. Problem Statement

Take a working Selenium login script and restructure it using the Page Object Model (POM). Create separate page classes containing locator definitions and UI methods, while keeping test assertions outside the page classes.

2. Objective

To demonstrate a maintainable Selenium structure in which page-specific locators and UI operations are separated from the actual test validation.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Google Chrome and ChromeDriver
- SauceDemo practice application
- Page Object Model (POM)
- Locators: By.ID, By.NAME, By.XPATH
- Assertions and page validation
- `time.sleep()` for visible execution during this assignment

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class LoginPage:
    URL = "https://www.saucedemo.com/"
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.NAME, "password")
    LOGIN_BUTTON = (
        By.XPATH,
        "//*[@type='submit' and @value='Login']"
    )

    def __init__(self, driver):
        self.driver = driver

    def open(self):
        self.driver.get(self.URL)
        time.sleep(3)

    def enter_username(self, username):
        self.driver.find_element(*self.USERNAME).send_keys(username)
        print("Username entered using POM.")
        time.sleep(3)

    def enter_password(self, password):
        self.driver.find_element(*self.PASSWORD).send_keys(password)
        print("Password entered using POM.")
        time.sleep(3)

    def click_login(self):
        self.driver.find_element(*self.LOGIN_BUTTON).click()
        print("Login button clicked.")
```

```

        time.sleep(5)

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

class DashboardPage:
    INVENTORY_CONTAINER = (By.ID, "inventory_container")

    def __init__(self, driver):
        self.driver = driver

    def get_current_url(self):
        return self.driver.current_url

    def is_inventory_page_displayed(self):
        time.sleep(3)
        return self.driver.find_element(
            *self.INVENTORY_CONTAINER
        ).is_displayed()

driver = webdriver.Chrome()

try:
    driver.maximize_window()

    login_page = LoginPage(driver)
    dashboard_page = DashboardPage(driver)

    print("Opening SauceDemo login page...")
    login_page.open()
    login_page.login("standard_user", "secret_sauce")

    print("Checking dashboard...")
    assert "/inventory.html" in dashboard_page.get_current_url()
    assert dashboard_page.is_inventory_page_displayed()

    print("Login successful!")
    print("Current URL:", dashboard_page.get_current_url())
    print("Dashboard displayed successfully.")
    print("Assignment 7 completed successfully.")

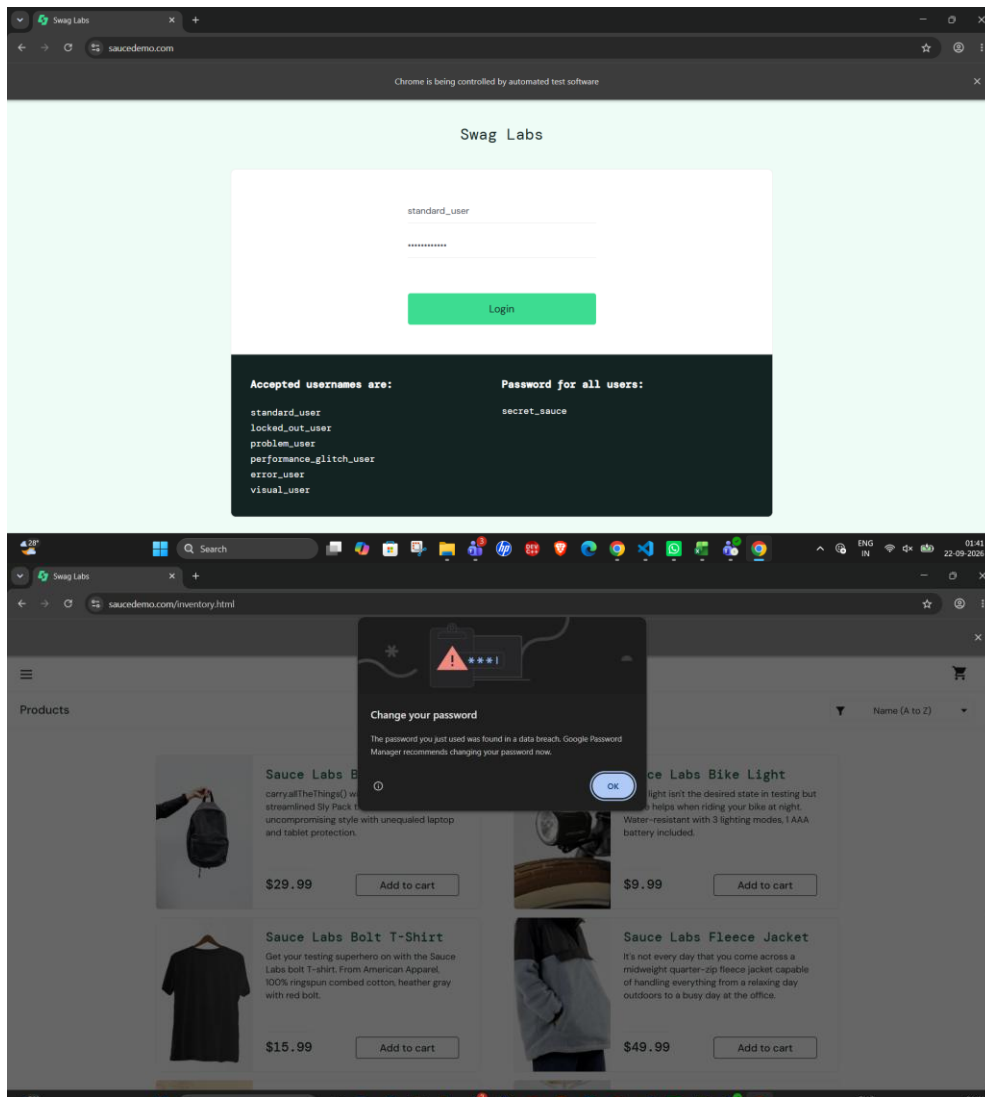
    time.sleep(5)

finally:
    driver.quit()

```

5. Output

The login page opened, the username and password were entered through the LoginPage object, and the dashboard was validated through the DashboardPage object.



6. Result

The POM-based login automation completed successfully and the expected inventory page was reached.

7. Observation

The LoginPage and DashboardPage classes contain locators and UI-related methods, while the assertions remain outside the page classes.

8. Conclusion

The experiment demonstrated how Page Object Model separates page interaction from test validation and improves code organization.

Experiment 8 — Data-Driven Automation (DDT)

1. Problem Statement

Create a Selenium login automation script that reads multiple test cases from an external CSV file. Execute the same login flow with correct and incorrect username/password combinations and validate the expected result.

2. Objective

To demonstrate data-driven testing by separating test data from automation logic and executing multiple scenarios using the same Selenium workflow.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Pandas
- CSV test data
- Google Chrome and ChromeDriver
- Data-driven testing
- Assertions and validation

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import pandas as pd
import time
import os

file_path = os.path.join(
    os.path.dirname(__file__),
    "login_test_data.csv"
)

test_data = pd.read_csv(file_path)

print("Test data loaded successfully.")
print("\nNumber of test cases:", len(test_data))

driver = webdriver.Chrome()

try:
    driver.maximize_window()

    for index, row in test_data.iterrows():

        test_case = row["test_case"]
        username = row["username"]
        password = row["password"]
        expected_result = row["expected_result"]
        expected_error = row["expected_error"]

        print("\n-----")
        print("Test Case:", test_case)
        print("Username:", username)
        print("Password:", password)
        print("-----")
```

```

driver.get("https://www.saucedemo.com/")
time.sleep(3)

username_field = driver.find_element(By.ID, "user-name")
password_field = driver.find_element(By.NAME, "password")
login_button = driver.find_element(
    By.XPATH,
    "//input[@type='submit' and @value='Login']"
)

username_field.send_keys(username)
print("Username entered.")
time.sleep(2)

password_field.send_keys(password)
print("Password entered.")
time.sleep(2)

login_button.click()
print("Login button clicked.")
time.sleep(3)

if expected_result == "success":
    assert "/inventory.html" in driver.current_url
    print("Expected result: Successful login")
    print("Actual result: Login successful")
    print("Test case passed.")

else:
    error_message = driver.find_element(
        By.CSS_SELECTOR,
        "h3[data-test='error']"
    ).text

    print("Expected error:", expected_error)
    print("Actual error:", error_message)

    assert error_message == expected_error
    print("Validation error matched.")
    print("Test case passed.")

time.sleep(3)

print("\n=====")
print("All data-driven test cases passed.")
print("Assignment 8 completed successfully.")
print("=====")

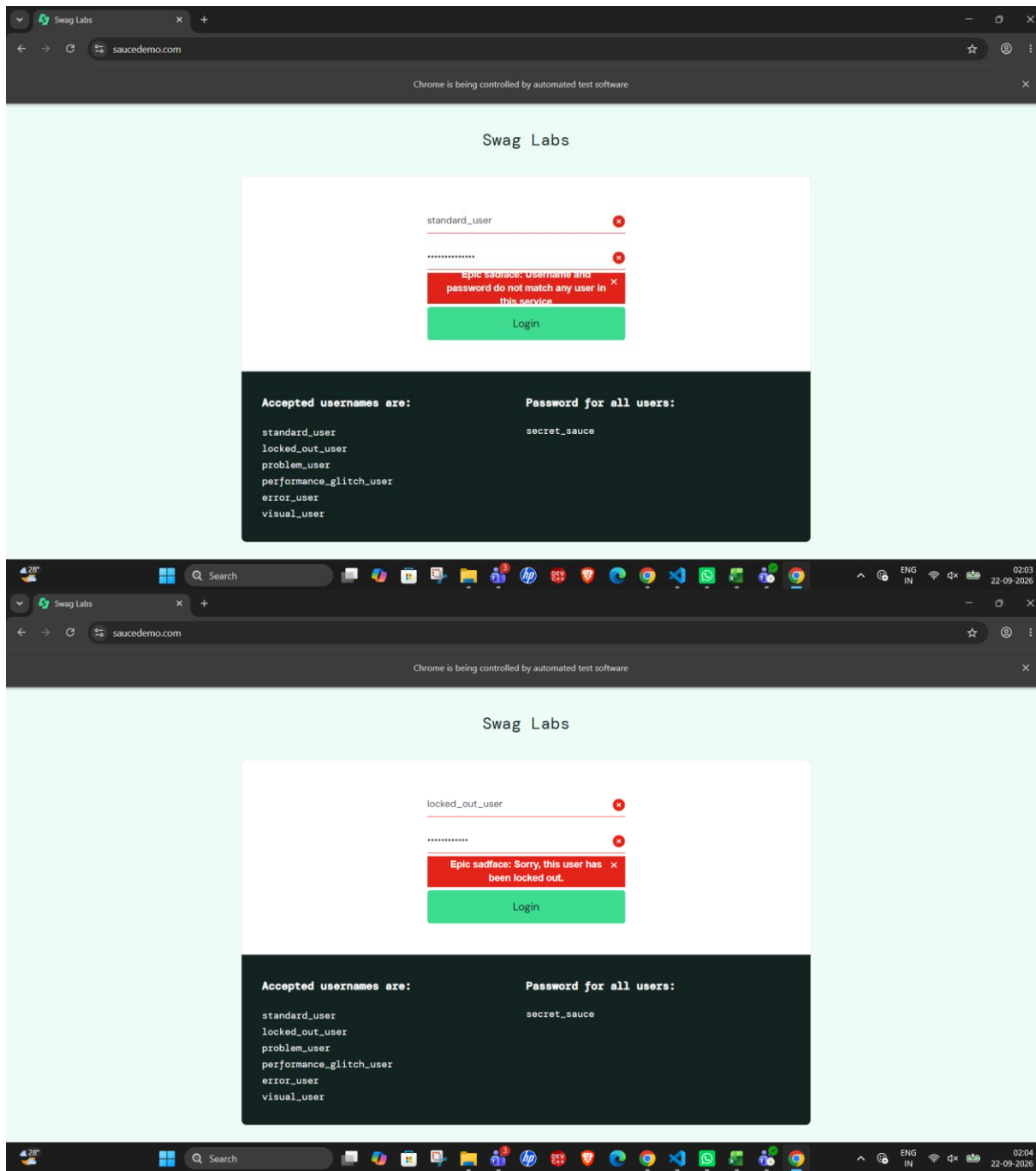
time.sleep(5)

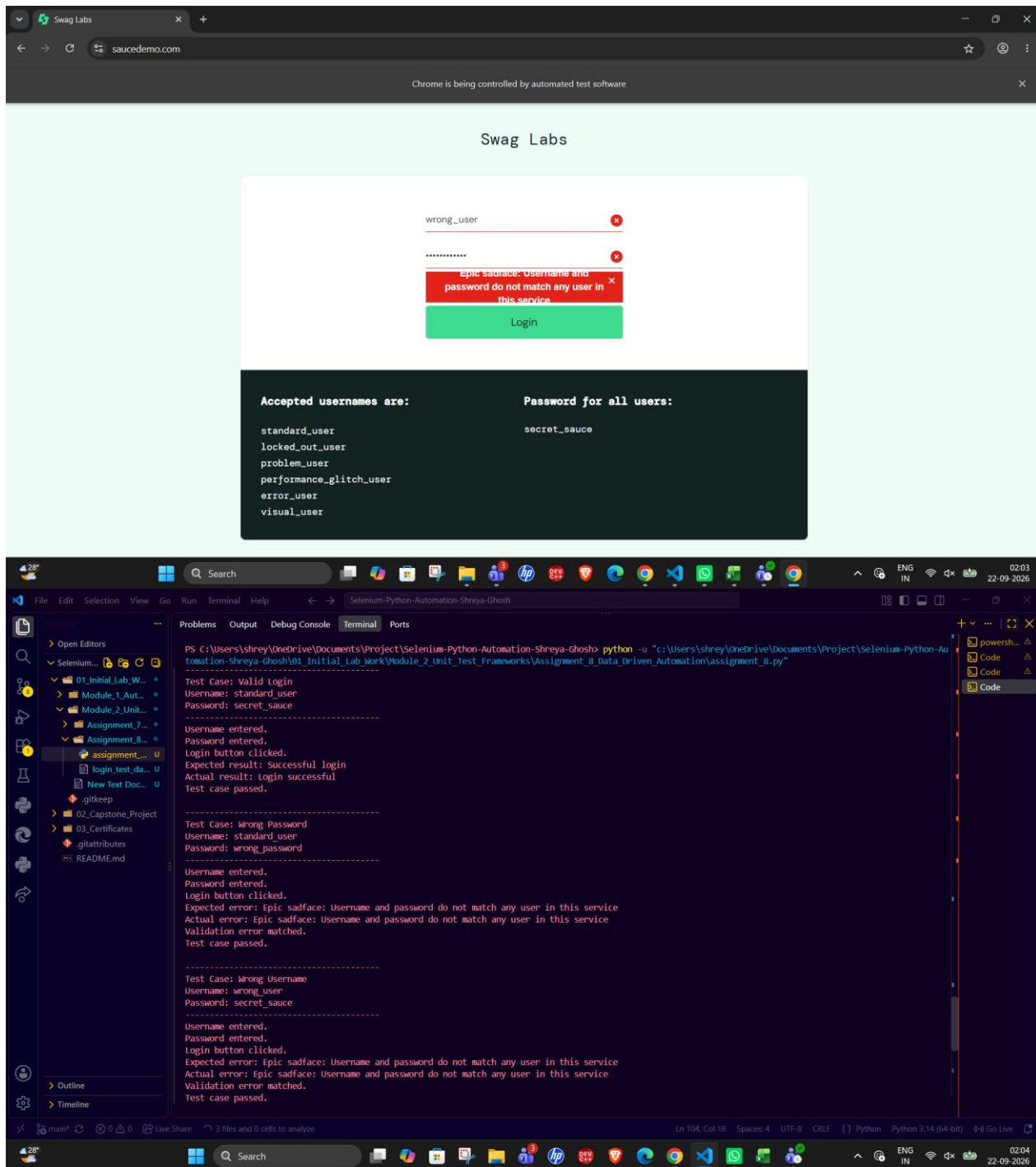
finally:
    driver.quit()

```

5. Output

The script reads login_test_data.csv and loops through the supplied test cases. Successful login is validated using the inventory URL, while invalid cases are validated against the displayed error message.





6. Result

The login automation is structured as a data-driven test using an external CSV file.

7. Observation

Separating test data from the automation script allows additional login scenarios to be added without rewriting the main Selenium flow.

8. Conclusion

The experiment demonstrated how external test data can drive repeated Selenium test execution and validation.

Experiment 9 — pytest Integration with HTML Reporting

1. Problem Statement

Convert the Selenium login automation into a pytest-based test suite. Use a pytest fixture for browser initialization and teardown, execute the suite from the terminal, and generate an HTML execution report.

2. Objective

To demonstrate Selenium integration with pytest, reusable browser setup and teardown through fixtures, and HTML test reporting.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- pytest
- pytest-html
- pytest-metadata
- Google Chrome and ChromeDriver
- pytest fixtures
- HTML reporting

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium.webdriver.common.by import By
import time

def test_valid_login(driver):
    driver.get("https://www.saucedemo.com/")
    time.sleep(2)

    username = driver.find_element(By.ID, "user-name")
    password = driver.find_element(By.NAME, "password")
    login_button = driver.find_element(
        By.XPATH,
        "//input[@type='submit' and @value='Login']"
    )

    username.send_keys("standard_user")
    time.sleep(1)

    password.send_keys("secret_sauce")
    time.sleep(1)

    login_button.click()
    time.sleep(3)

    assert "/inventory.html" in driver.current_url
```

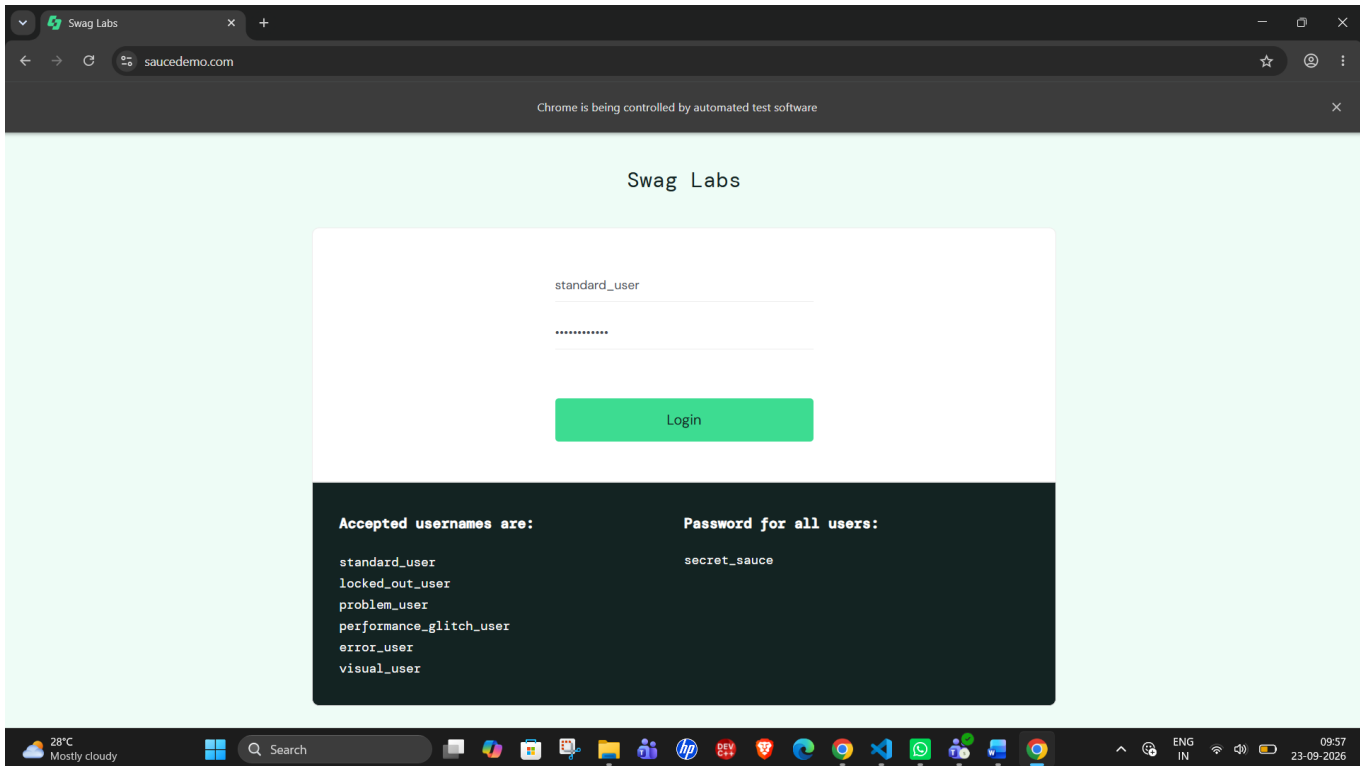
```
print("Valid login test passed.")

def test_login_page_title(driver):
    driver.get("https://www.saucedemo.com/")
    time.sleep(3)

    assert "Swag Labs" in driver.title
    print("Login page title test passed.")
```

5. Output

The PyTest suite executed two tests successfully. The HTML report was generated using pytest-html with the --self-contained-html option.



Swag Labs

Products

Name (A to Z)



Sauce Labs Backpack

carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection.

\$29.99

Add to cart



Sauce Labs Bike Light

A red light isn't the desired state in testing but it sure helps when riding your bike at night. Water-resistant with 3 lighting modes, 1 AAA battery included.

\$9.99

Add to cart



Sauce Labs Bolt T-Shirt

Get your testing superhero on with the Sauce Labs bolt T-shirt. From American Apparel, 100% ringspun combed cotton, heather gray with red bolt.

\$15.99

Add to cart



Sauce Labs Fleece Jacket

It's not every day that you come across a midweight quarter-zip fleece jacket capable of handling everything from a relaxing day outdoors to a busy day at the office.

\$49.99

Add to cart

Selenium-Python-Automation-Shreya-Ghosh

File

Edit

Selection

View

Go

Run

Terminal

Help

←

→

01_Initial_Lab_Work

Module_2_Unit_Test_Frameworks

Assignment_9_PyTest_HTML_Reporting

test_login.py

test_login_page_title

37

print("Valid login test passed.")

38

39

40

def test_login_page_title(driver):

41

42

driver.get("https://www.saucedemo.com/")

43

44

time.sleep(3)

45

46

assert "Swag Labs" in driver.title

47

Problems

Output

Debug Console

Terminal

Ports

(.venv) PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh> python -m pytest -v

===== test session starts =====

platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\.venv\Scripts\python.exe

cachedir: .pytest_cache

metadata: {'Python': '3.14.3', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}

rootdir: C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh

plugins: html-4.2.0, metadata-3.1.1

collected 2 items

01_Initial_Lab_Work\Module_2_Unit_Test_Frameworks\Assignment_9_PyTest_HTML_Reporting\test_login.py::test_valid_login PASSED [50%]

01_Initial_Lab_Work\Module_2_Unit_Test_Frameworks\Assignment_9_PyTest_HTML_Reporting\test_login.py::test_login_page_title PASSED [100%]

===== 2 passed in 23.64s =====

(.venv) PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh>

6. Result

The Selenium tests were successfully integrated with PyTest and the browser lifecycle was handled through a fixture.

7. Observation

The fixture provides centralized browser setup and teardown, while PyTest organizes the tests and pytest-html generates a readable execution report.

8. Conclusion

The experiment demonstrated structured Selenium test execution using PyTest and HTML reporting.

Fixture File — conftest.py

Module 2 — Unit Test Frameworks | 11

```

import pytest
from selenium import webdriver
import os

@pytest.fixture
def driver(request):

    driver = webdriver.Chrome()
    driver.maximize_window()

    yield driver

    report = getattr(request.node, "rep_call", None)

    if report and report.failed:

        os.makedirs("screenshots", exist_ok=True)

        screenshot_path = os.path.join(
            "screenshots",
            f"{request.node.name}.png"
        )

        driver.save_screenshot(screenshot_path)

    driver.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):

    outcome = yield
    report = outcome.get_result()

    setattr(item, "rep_" + report.when, report)

```

Terminal Commands

```

python -m pytest -v
python -m pytest -v --html=report.html --self-contained-html

```